

ECSPros — Dayanıklılık, Performans ve Ölçeklenebilirlik Değerlendirmesi

15 Ağustos 2026 · Statik kod analizi + yapılandırma incelemesi

Tarih: 2026-08-15 **Kapsam:** Backend (.NET 8 Modular Monolith), veritabanı, önbellekleme, altyapı, güvenlik, çoklu sunucu hazırlığı **Yöntem:** Statik kod analizi + yapılandırma incelemesi (kaynak `src/`, `docker-compose.yml`, nginx konfigürasyonu, git izleme durumu) + **canlı ortam ölçümü** (2026-08-16, §3.9). Tam yük testi (tepe yük, eşzamanlılık) hâlâ yapılmadı.

Not: Bu belge mevcut durumu tespit eder, sıralı bir iyileştirme yol haritası önerir. Yol haritasındaki hiçbir değişiklik bu değerlendirme sırasında uygulanmamıştır. **Tamamlayıcı rapor:** Aynı klasördeki `ECSPros-Kod-Optimizasyon-Raporu.pdf` (15 Ağustos 2026) yalnızca uygulama kodu performansına odaklanır; API/PostgreSQL/Redis'in **ayrı sunucularda** çalıştığı dağıtık yerleşim ve **4.000 eşzamanlı kullanıcı** hedefiyle hazırlanmıştır. Bu belgenin §3.1, §3.2 ve Faz 2 bölümleri o raporun P0/P1/P2 bulgularıyla bütünleştirilmiştir.

1. Yönetici Özeti

ECSPros, **ürün/iş mantığı açısından olgun** bir kod tabanıdır: modüler monolit sınırları iyi çizilmiş, CQRS (MediatR) tutarlı kullanılmış, `Result<T>` deseni yaygın, çapraz modül bağımlılıkları arayüzlerle (`IProductService`, `IStockService`, `ICacheService` vb.) gevşetilmiş, sipariş numarası üretimi ve hesap bakiyesi güncellemesi gibi kritik noktalarda atomik SQL / advisory lock kullanılmış, Redis cache hata-güvenli tasarlanmış.

Buna karşılık **operasyonel dayanıklılık ve yatay ölçeklenebilirlik katmanı erken aşamadır**. Sistem bugün **tek sunucu, tek process** varsayımıyla tasarlanmış ve test kullanımı için yeterlidir; ancak:

- Eşzamanlı stok güncellemeleri "read-modify-write" ile yapılıyor** — iyimser kilit (concurrency token), koşullu `UPDATE` veya satır kilidi yok. Trafik arttıkça **fazla satış (oversell) / kayıp güncelleme** riski gerçektir.
- EF Core `EnableRetryOnFailure` hiçbir yerde açık değil** — geçici DB hatası/deadlock anında 500 olarak isteğe döner, otomatik yeniden deneme yok.
- `/health` ucu ve health check altyapısı yok** — nginx/yük dengeleyici/izleme açısından canlılık-hazırlık sinyali eksik.
- Sırlar (DB/Redis/JWT şifreleri) git'e işlenmiş** — `appsettings.json`, `docker-compose.yml` ve 12 adet `*DbContextFactory.cs` düz metin kimlik bilgisi içeriyor.
- Çoklu sunucuya geçiş bugün kırılr:** SignalR backplane'siz, Data Protection anahtarları sunucu-yerel dosyada, rate limiter süreç-içi, arka plan worker'ları leader-election'sız, migration'lar açılışta yarışa açık.

Genel değerlendirme: Tek instance'lık test/erken üretim için "iyi"; gerçek yüke ve 2+ instance'a çıkmadan önce **Faz 0 (acil düzeltme) ve Faz 1 (dayanıklılık/güvenlik temelleri)** mutlaka tamamlanmalı. Performans ve ölçekleme iyileştirmeleri **Faz 2-3'te** planlanabilir.

2. Mevcut Durum

2.1 Mimari Genel Bakış

- **Model:** .NET 8 Modular Monolith — 15 modül (Accounts, Catalog, Cms, Core, Crm, Finance, Fulfillment, Iam, Integration, Inventory, Order, Pos, Promotion, Requests, Storefront), tek API host (`src/ECSPros.Api`).
- **Katmanlar:** Her modül `Domain / Application / Infrastructure`. Application katmanı CQRS (Commands/Queries + MediatR handler), Infrastructure EF Core + migration.
- **Veri erişimi:** 15 ayrı `DbContext`, tek PostgreSQL 16 (`ecommerce_db`), `NpgsqlDataSource` singleton + `EnableDynamicJson()`.
- **Cache:** `ICacheService` (Redis — `StackExchange.Redis`) + `IMemoryCache` (süreç-içi).
- **Auth:** JWT HS256 (symmetric), refresh token rotation (`UserSessions`, SHA256 hash), `type` claim'i ile üye/api_client/supplier_user/admin ayrımı.
- **Olay akışı:** MediatR domain event'leri süreç-içi (in-process `IPublisher`); kargo bildirimi ve eski sistem sipariş senkronu için iki ayrı **outbox** tablosu.
- **Deploy:** Tek systemd servisi (`ecspros`), nginx + postgres + redis Docker Compose ile aynı sunucuda (`51.178.208.59`). API `localhost:5000` üzerinden nginx'e proxy'lenir.

2.2 Güçlü Yönler (korunması gerekenler)

- **Modüler sınırlar temiz** — çapraz modül erişimi arayüz sözleşmeleriyle (`src/Shared/ECSPros.Shared.Contracts/`) gevşetilmiş; doğrudan modül-arası `DbContext` referansı az.
- ``Result<T>`` + **FluentValidation pipeline** — handler'larda tutarlı hata modeli (`ValidationBehavior`).
- **Atomik sipariş numarası üretimi** — `OrderNumberService.GenerateAsync` tek `UPDATE ... RETURNING` ile sayaç ilerletiyor (`ON CONFLICT DO NOTHING` ile yarışta seri açma). Eşzamanlı iki checkout aynı numarayı alamaz.
- **Hesap bakiyesi serileştirme** — `PostAccountTransactionCommand` `pg_advisory_xact_lock` ile sahip-hesap üzerinde kilitliyor; eski sistem görsel senkronunda da `pg_try_advisory_xact_lock` kullanılıyor.
- **Redis hata-güvenli** — `RedisCacheService` hiçbir durumda exception fırlatmaz; devre kesici (2 dk) + kısa timeout'lar (`AbortOnConnectFail=false`, 1-1.5 sn) kodda zorlanır; Redis yoksa `NoOpCacheService` devreye girer, site cache'siz çalışır.
- **Hot-path index'ler iyi** — Sipariş (`Status, CreatedAt`) partial index'leri, Storefront `FirmPlatformId+Status / +ProductCode+Status` bileşik index'leri, Catalog benzersiz Barcode/Sku/Slug index'leri mevcut.
- **Sıralı hız sınırlama katmanları** — nginx `limit_req` (60r/m auth, 30r/s genel) + süreç-içi `AddRateLimiter` (storefront auth/sensitive uçlarında).
- **Statik varlık önbellegi** — `?v=` sürümlü CSS/JS 1 yıl immutable, HTML no-cache (tarayıcı bayat vitrin tutmaz).

2.3 Yük Bağlamı (referans)

- `docs/performans-analizi.md` (2026-05) projeksiyonu: orta ölçekli e-ticarette tepe ~20-30 RPS, yüksek ölçekte ~200-300 RPS. Bu belge "doğru yapılandırılmış tek sunucu orta yükü rahat kaldırır, ancak çoklu-instance için hazırlık gerekir" önermesini doğrular.

- ECSPros-Kod-Optimizasyon-Raporu.pdf (2026-08-15) hedefi: **4.000 eşzamanlı kullanıcı**, API/PostgreSQL/Redis'in **ayrı sunucularda** çalıştığı dağıtık yerleşim. Bu yerleşimde ana kaldıraç cache boyutu değil, **istek başına üretilen veri hacmi ve PostgreSQL ağ turu sayısıdır**.

3. Bulgular

Her bulgu **Şiddet** | **Alan** | **Dosya:Satır** biçimindedir. Şiddet: **Kritik** / **Yüksek** / **Orta** / **Düşük**.

3.1 Performans ve Veritabanı

- **Kritik** | ``GetStoreProductsQuery`` tüm platform fiyatlarını çekiyor | ``StorefrontChannelPricingService.GetActiveVariantPricesAsync``
Kod optimizasyon raporunun bir numaralı darboğazı: kullanıcı sayfada yalnızca ~24 ürün göreceksin olsa da platformdaki **bütün aktif varyant fiyatları** belleğe alınıp API'ye taşınıyor. Fiyat çağrısı sayfadaki **variantIds** kümesiyle sınırlanmalı.
- **Yüksek** | **Bellekte sıralama** | ``GetStoreProductsQuery`` / ``GetChannelCategoryProducts``
Fiyat, metrik ve arama sıralamalarında tüm adaylar **ToListAsync** ile belleğe alınıp orada sıralanıyor; katalog büyüdükçe CPU/RAM artar. Sıralama + sayfalama **ORDER BY ... OFFSET/LIMIT** ile PostgreSQL'de yapılmalı; uzun vadede efektif fiyat/puan/yorum/satış/stok taşıyan ürün arama read-model'i kurulmalı.
- **Yüksek** | **Kartezyen çarpım** | ``src/Modules/Catalog/ECSPros.Catalog.Application/Queries/GetProductDetail/GetProductDetailQueryHandler.cs:17-21``
Ürün detayında **Include(Attributes)**, **Include(Variants).ThenInclude(VariantAttributes/Images)** gibi **birden çok koleksiyon** **Include** ediliyor; **AsSplitQuery()** yok. **Attributes × Variants × VariantAttributes × Images** satır patlaması üretir. Kamusal ürün detay sayfasının en kritik sorgusu.
- **Yüksek** | **Kartezyen çarpım** | ``src/Modules/Order/ECSPros.Order.Application/Queries/GetOrderDetail/GetOrderDetailQueryHandler.cs:17-21``
Include(Items) + Include(Payments) kardeş koleksiyonlar; **AsSplitQuery/AsNoTracking** yok.
- **Yüksek** | **Kartezyen çarpım** | ``src/Modules/Iam/ECSPros.Iam.Application/Queries/GetUserDetail/GetUserDetailQueryHandler.cs:35-37``
Include(UserRoles) + Include(UserPermissions) kardeş koleksiyonlar.
- **Orta** | **Kartezyen çarpım (ek)** — aynı desen: `GetReturnDetailQueryHandler.cs:20-21`, `GetFirmDetailQuery.cs:37-39`, `GetProductGroupsQuery.cs:53-56`, `GetAttributeTypesQuery.cs:53-54`, `Promotion/ProductCampaignResolver.cs:21-24`, `GetSessionSummaryQueryHandler.cs:20`, `ApproveProductSubmissionCommandHandler.cs:36`.
Genel: Repo genelinde **AsSplitQuery()** **sıfır** kullanım. Birden çok koleksiyon include eden her sorguda bu desen tekrarlanıyor.
- **Orta** | ``AsNoTracking`` **eksik** | ``GetProductDetailQueryHandler.cs:17`` ve benzer **read-only handler'lar**
Salt-okunur handler'lar tam entity grafini change-tracker'a sokuyor; gereksiz bellek/CPU. **AsNoTracking()** yalnızca DTO projeksiyonlu bazı handler'larda tutarlı değil.
- **Orta** | **Arama döngüsünde N+1** | ``src/Modules/Storefront/.../GetChannelCategoryProductsQuery.cs:427-433`` ve ``Catalog/GetStoreProductsQuery.cs:158-165``
Arama kelimeleri üzerinde **foreach** içinde her kelime için ayrı **await ...ToListAsync()** — her kelime başına ek round-trip.
- **Orta** | **Bellek içi `Contains` filtresi** | `Storefront `GetProductsLeafChannelCategories` / `GetChannelCategoryFacets``
Satışa-açık ürün ID kümesi belleğe alınıp **ProductIds.Contains(p.Id)** ile filtre ediliyor; büyük kümelerde **SQL yerine bellek içi tarama**.

- **Orta | Eksik index'ler (spot kontrol)** — `Orders.MemberId` (üye sipariş geçmişi), `OrderItems.SupplierId` (tedarikçi filtrelemesi), `Members(Status, CreatedAt)` (admin üye listesi) için index yok. Hot-path index'ler iyi olsa da bu alanlar yük altında yavaşlar.
- **Düşük/İyi | Pagination** — Liste uçlarının büyük kısmı `PagedResult` ile sayfalı; yalnızca küçük lookup/config tabloları sınırsız okunuyor. Kabul edilebilir.
- **Düşük | Storefront liste tur sayısı** — Kategori sayfası 30+ ayrı round-trip üretebiliyor (stok/fiyat/sosyal kanıt/metrik zenginleştirmeleri). Tek tek ucuz ama birleştirilebilir.

3.2 Ön bellekleme

- **Orta | Redis hit yolunda DB çağrısı | `PageComposer.cs` (ana sayfa kompozisyonu)**
`PageComposer.ComposeAsync` cache anahtarını üretmek için önce PostgreSQL'den aktif snapshot version'ını okuyor; hazır veri Redis'te olsa bile her istekte DB'ye gidiyor. Aktif snapshot (`page-active:{platformId}`) kısa ömürlü cache'te tutulup yayında güncellenmeli.
- **Orta | Yalnızca TTL ile geçersizleştirme |**
``GetChannelCategoryProductsQuery.cs:126-147``, ``GetChannelCategoryFacetsQuery.cs:57-165``, ``PageComposer.cs:66-225``
Kategori ürün listesi (`channelcat:products:v11...`, TTL 10 dk) ve facet'ler (`channelcat:facets:v9...`, TTL 10 dk) **yazma anında geçersizleştirilmiyor** — yalnızca TTL ile bayatlıyor. Ürün/fiyat/stok/kategori güncellemesi **10 dakikaya kadar** bayat görünür. Ana sayfa blokları (`page:c5:...`, 5 dk) yayınlamada `v{Version}/{SnapshotId}` anahtar değişimiyle geçersizleştiriyor (bu kısım iyi).
- **Orta | Cache stampede (thundering herd) | yukarıdaki handler'lar +**
``InStockProductProvider.cs:47-57``, ``EffectivePriceProvider.cs:52-66``
Cache-miss popülasyonu kilitsiz `GetAsync` → `hesapla` → `SetAsync`;
`IMemoryCache.GetOrCreateAsync` da eşzamanlı factory'leri tekilleştirmez. Soğuk başlatma/TTL dolumunda aynı ağır sorgu çok sayıda eşzamanlı çalışır → DB'ye ani yük. Çözüm: süreç-içi key-bazlı `SemaphoreSlim` + node'lar arası kısa Redis kilidi + stale-while-revalidate.
- **Düşük (gizli tuzak) | `RemoveByPatternAsync` no-op |**
``src/Shared/ECSPros.Shared.Infrastructure/Caching/RedisCacheService.cs:110-116``
Metot gövdesi `return Task.CompletedTask;` — desen bazlı silme sessizce hiçbir şey yapmıyor. Bugün **çağıranı yok**, dolayısıyla aktif bir hata yok; ama gelecekte kullanılırsa geçersizleştirme sessizce çalışmaz.
- **Orta | Süreç-içi cache çoklu-instance tutarsızlığı | nav/footer/legal**
`(`StorePageController.cs:91-123`)`
Navigasyon/footer/yasal metin `IMemoryCache` (5 dk) — instance başına ayrı; çoklu-instance'ta her instance yeniden doldurur, admin değişikliği $N \times \text{TTL}$ kadar gecikir.
- **Düşük | Anahtar kapsamı | `InStockProductProvider.cs:14-15`**
`in-stock-product-ids / in-stock-variant-ids` anahtarları platform-agnostik (`FirmPlatformId` içermiyor). Bugün veri de global kapsamlı olduğundan etkisiz; çok kiracılık gelirse paylaşım riski.

Özet: Redis üretimde gerçek hot-path'lerde (kategori listesi, facet, ana sayfa blokları) aktif; ancak geçersizleştirme stratejisi **TTL-ağırlıklı** ve yazma-yolu ile bağlantısız. "En fazla 10 dk bayat" kabul edilebilirse bugünkü hâli yeterli; aksi hâlde event-tabanlı invalidasyon gerekir.

3.3 Dayanıklılık

- **Yüksek | DB retry yok | tüm modül `DependencyInjection.cs` (örn.**
``Order/.../DependencyInjection.cs:13-15`)`
15 DbContext'in tamamı `UseNpgsql(dataSource, ...)` ile kaydediliyor, `EnableRetryOnFailure()` **hiçbirinde** yok (repo geneli 0 eşleşme). Geçici bağlantı kopması/deadlock anında istek başarısız döner.

- **Yüksek | Polly yardımcısı kullanılmıyor |**
`src/Shared/ECSPros.Shared.Infrastructure/Http/ResilientHttpClientExtensions.cs:13-27`
`AddResilientHttpClient` (retry 3 + circuit breaker) tanımlı ama **hiçbir çağrı yeri yok** (ölü kod).
Diş çağrılarının tamamı retry/backoff'suz.
- **Yüksek | Kayıtlı olmayan named HttpClient |** `GorselAramaController.cs:49,107`,
`UrunListesiController.cs:221`, `DeviceAttestationServices.cs:59,129`,
`TrendyolSellerClient.cs:64`, `TrendyolReferenceDownloader.cs:25`
`CreateClient("visual-search")`, `"play-integrity"`, `"TrendyolSeller"`,
`"TrendyolReference"` kullanılıyor ama bu isimlerle `AddHttpClient(name)` **kaydı yok** (yalnızca
`"paytr"` ve `"legacy-order"` var). `IHttpClientFactory.CreateClient(name)` kayıtsız isimde
exception fırlatır. Bu özellikler şu an aktif/konfigüre olmadığı için gizli (latent) bir kusurdur;
açıldığında çalışma anında patlar.
- **Orta | HttpClient timeout tutarsız |** `Program.cs:183` (`paytr`, süre yok),
`Shared/.../DependencyInjection.cs:53` (SMS),
`Integration/.../DependencyInjection.cs:39` (adaptörler)
Çoğu named/default client timeout'suz (varsayılan 100 sn). Yalnızca `legacy-order` (30 sn),
`FaturaPdfProxy` (20 sn), `PayTrDirectService` (15 sn, çağrı yerinde) süre belirliyor. Asılı bir
ödeme/SMS/kargo ucu thread'i 100 sn'ye kadar bloklayabilir.
- **Yüksek | Health check yok | tüm repo**
`AddHealthChecks` / `MapHealthChecks` / `/health` hiç yok. Nginx/yük dengeleyici canlılık-hazırlık
sinyali alamaz; DB/Redis bağımlılık kontrolü de yok.
- **Orta | Graceful shutdown eksik |** `Program.cs:54`
Yalnızca `BackgroundServiceExceptionBehavior = Ignore` var (worker hatası host'u çökertmez —
iyi) ama `ShutdownTimeout` tanımsız. Ayrıca `Ignore` davranışı sessizce ölen worker'ı yüzeye
çıkarmaz (gözlemlenebilirlik açığı).
- **Düşük | Hata modeli basit |** `Middleware/GlobalExceptionHandler.cs`
Tüm istisnaları yakalar, genel 500 döner; correlation ID, ProblemDetails, hassas veri filtreleme yok.
`InvalidOperationException` → 400 eşlemesi bazı iç hataları yanlış sınıflandırabilir.

3.4 Arka Plan İşleri ve Transactional Bütünlük

- **Yüksek | Worker çoğaltması (çoklu-instance) |** `SettlementEligibilityWorker.cs:40-75`
`Status=="pending" && EligibleAt<=now` satırlarını kilitsiz seçip
`PostAccountTransactionCommand` ile hesaba yazıyor. İki instance çalışırsa **komisyon/ledger çift yazılır** (para çoğaltması).
- **Yüksek | Worker çoğaltması |** `Fulfillment/CargoNotifyWorker.cs:46-50`
Pending outbox satırlarını `Take(20)` ile atomik "claim" olmadan okuyup kargo API'sine gönderiyor.
İki instance çift kargo bildirimi gönderir (benzersiz `PackageId` index'i DB'de çift satırı önler ama çift
API çağrısını önlemez).
- **Orta | Worker çoğaltması |** `LegacySyncWorker.cs:66-75`,
`LegacyOrderSyncService.cs:60-66`, `SavedSearchNotifier.cs:32-85`
Aynı sipariş iki instance tarafından eski sisteme POST edilebilir; favori arama bildirimi çift e-posta
gönderebilir.
- **İyi (referans) | Çoklu-instance koruması örneği |**
`MarketplaceReferenceSyncService.cs:53-80`
`ConcurrentDictionary` + DB `mp_sync_runs status='running'` + bayat heartbeat kapatma —
worker'lar arasında koruma olduğunu gösteren tek örnek. Diğer worker'lar bu desene taşınmalı.
- **Yüksek | Domain event'leri transaction'sız |**
`CompleteSaleCommandHandler.cs:96-101`, `RefundSaleCommandHandler.cs:43-48`,
`MarkDeliveredCommandHandler.cs:50-53`, `MarkShippedCommandHandler.cs:55-58`
`SaveChangesAsync()` commit olduktan sonra MediatR `Publish` ile event'ler süreç-içi dağıtılıyor;
cevreleyen transaction yok. Commit ile event yan etkisi (stok düşme, SignalR bildirimi) arasında
çökme olursa durum kısmı/kayıp kalır. (Kargo bildirimi ve eski sistem siparişi için ayrı outbox
tabloları var — bu iyi; ancak ana stok/ödeme akışları outbox'sız.)

- **İyi | Outbox koruması | kargo + legacy order outbox'ları** benzersiz index'lerle korunuyor; bu desen diğer kritik akışlara yaygınlaştırılmalı.

3.5 Eşzamanlılık (Concurrency)

- **Kritik | Stok fazla satışı / kayıp güncelleme |**
``src/Modules/Inventory/ECSPros.Inventory.Application/Services/StockOps.cs:58-71,112-136,143-150``
`ReserveAsync / ConsumeAsync` stok satırlarını okuyup bellekte `Quantity/ReservedQuantity` değiştiriyor, `SaveChanges` ile yazıyor. **Koşullu `UPDATE ... WHERE Quantity >= x` yok, satır kilidi yok, `RowVersion`/`IsConcurrencyToken` yok** (repo geneli `FOR UPDATE/RowVersion = 0`). İki eşzamanlı istek aynı stoğu iki kez düşebilir (last-write-wins). E-ticarete en kritik düzeltme kalemi.
- **İyi | Sipariş numarası | `OrderNumberService.cs` — `UPDATE...RETURNING` atomik** (yukarıda).
- **İyi | Hesap bakiyesi | `PostAccountTransactionCommand.cs:57,134` —**
`pg_advisory_xact_lock` ile serileşiyor.

3.6 Ölçeklenebilirlik / Çoklu Sunucu

Genel: Sistem bugün **tek instance** varsayımıyla yazılmış. 2+ instance'a geçiş aşağıdakiler olmadan kırılır:

- **Yüksek | SignalR backplane yok | `Program.cs:207`**
`AddSignalR()` Redis backplane'siz. Hub grupları ve bağlantılar instance başına; A instance'ında yayınlanan olay B instance'ındaki istemciye ulaşmaz. WebSocket için nginx'te sticky (ip_hash) de yok (`docker/nginx/conf.d/locations.inc` tek `proxy_pass`).
- **Yüksek | Data Protection anahtarları sunucu-yerel | `Program.cs:131-136`**
`PersistKeysToFileSystem(~/.ecsp/keys) + SetApplicationName` — paylaşımlı key ring yok (Redis/Azure KV/sertifika). Aynı makinelerde her instance kendi anahtarını üretir; A'da şifrelenen `FirmPlatformIntegration.Credentials` B'de çözülemez. (JWT HS256 simetrik olduğu için cookie/DP'ye bağlı değil — tek etkilenen şifreli kimlik bilgileri.)
- **Orta | Rate limiter süreç-içi | `Program.cs:392-417`**
`FixedWindowRateLimiter` IP-bazlı ama distributed store'suz; her instance kendi sayacını tutar → toplam izin Nx artar. (nginx katmanı birinci savunma; app-layer ikinci savunma olarak zayıf kalır.)
- **Orta | Açılıştaki migration/seed yarışması | `Program.cs:607-609` + `DatabaseSeeder.cs:747,875,900`**
`SeedAsync` her açılıştaki CMS/CRM/Storefront için `MigrateAsync()` çağırıyor. İki instance aynı anda başlarsa migration yarışması olur. Ayrıca diğer 12 modül açılıştaki migrate EDİLMİYOR (deploy'da manuel `dotnet ef database update` gerekiyor) — **tutarsız** bir strateji.
- **Orta | Worker leader-election yok | §3.4 — 8 worker'ın çoğu çift çalışır.**
- **Orta | Medya yerel disk | `Catalog/.../DependencyInjection.cs` → `LocalDiskImageUploadService`**
Yüklenen görseller `/opt/ECSP/AI/media` yerel diske yazılıyor (nginx mount ile servis ediliyor). Çoklu-instance/çoklu-makinede paylaşımlı depo (S3/NFS/object storage) gerekir.
- **Yüksek | Kimlik güvenlik sayaçları süreç-içi | `Crm/.../LoginMemberCommand.cs:31-56`, `DeviceAttestationServices.cs:195-215`**
Üye başarısız giriş kilidi `IMemoryCache` ile "tek host" varsayımına dayanıyor (5 deneme bütçesi N instance'ta Nx genişler); device-token secret'ı `IMemoryCache`'te — A'da üretilen token B'de reddedilir.
- **Düşük | Süreç-içi cache'ler | 54 `IMemoryCache` tüketicisi** — çoğu salt-okunur referans verisi (facet/attribute tipi vs.), instance başına yeniden hesaplama kabul edilebilir.

3.7 Güvenlik ve Konfigürasyon Hijyeni

- ****Kritik | Sırlar git'te | `src/ECSPros.Api/appsettings.json:19-52`, `docker-compose.yml`, `12x *DbContextFactory.cs:12**`**
Düz metin PostgreSQL şifresi, Redis şifresi ve `Jwt:Secret` git'te izlenen dosyalarda. `appsettings.Production.json` .gitignore'da (izlenmiyor) ama `appsettings.json` (base) izleniyor ve üretim değerlerini de içeriyor. `12 *DbContextFactory.cs` aynı DB şifresini tekrarlıyor.
- **Kritik | Tek paylaşılan HS256 secret | ``Program.cs:298-314`` + ``Iam/.../JwtTokenService.cs``**
Admin/üye/api_client/supplier_user token'larının tümü aynı simetrik ``Jwt:Secret`` ile imzalanıyor; kimlik sınıfları yalnızca kendi beyan ettiği `type` claim'i ile ayrışıyor. Tek secret sızarsa her kimlik taklit edilebilir.
- **Orta | Admin/supplier login rate-limit'siz | ``AuthController.cs:26,39,52``, ``SupplierAuthController.cs:18-48``**
Admin `login/token/refresh` ve supplier girişleri `[AllowAnonymous]` ama `[EnableRateLimiting]` yok. (nginx `limit_req` birinci savunma olarak var; app-layer'da eksik.)
- **Orta | Zayıf eski hash'ler | ``Crm/.../MemberPasswordHasher.cs:22-38``**
Üye şifresi doğrulamada eski tuzsuz MD5/SHA256 kabul ediliyor; BCrypt'e yükseltme yalnızca başarılı giriş sonrası. Her kullanıcı giriş yapana dek eski hash kırılabilir kalır.
- **Orta | Geniş CORS | ``Program.cs:366-372``, ``appsettings.Production.json:31-37``**
`AllowAnyMethod().AllowAnyHeader().AllowCredentials()`; prod origin listesinde `http://localhost:3000/5173` ve düz HTTP `http://51.178.208.59` var.
- **Orta | Data Protection anahtar yedeği | ``Program.cs:132-136``**
Anahtar halkası kaybedilirse DB'deki şifreli kimlik bilgileri çözülemez (kaynak yorumu da bunu söylüyor). Yedekleme/prosedür tanımlanmalı.
- **Düşük | Varsayılan admin + stdout'a şifre | ``DatabaseSeeder.cs:1100-1127``**
`admin` / bilinen şifre seed ediliyor ve `Console.WriteLine` ile log'a basılıyor (`MustChangePassword=true` kısmen hafifletiyor).
- **Düşük | Örnek env'de secret | ``tools/mobile/ecspros-staging.env.example``**
`MobileAttestation__DevBypassSecret` değeri örnek dosyada işlenmiş.
- **Düşük | Kök dizinde sahipsiz dosya | ``/opt/ECSProsAI/t-E6an3jYTHgSvvoQg``**
`ECSGYE.Solution.rar` için imzalı indirme linki içeren kayıtlı HTML sayfası (WeTransfer). Repo kökünde duruyor.
- **İyi | SQL injection yok (incelenen yollarda) — `FromSqlInterpolated` / parametrelili advisory lock kullanılıyor; kullanıcı girdisiyle string-concat `FromSqlRaw` bulunmadı.**
- **İyi | İç Swagger yalnız Development — `/swagger` (iç doküman) prod'da 404; `partner/mobile` dokümanları dış entegratörler için kasıtlı açık.**

3.8 Test ve Gözlemlenebilirlik

- **Yüksek | Otomatik test yok | ``src/ECSPros.sln``**
Ana projede `*Tests.cs` / `*Test.cs` sıfır (bulunan test projeleri gitignored `ECSGYE.Solution` eski inceleme kopyasına ait). Regresyon güvencesi, stok/ödeme gibi kritik akışlar için test yok.
- **Yüksek | Health check yok | §3.3 — `/health` ve bağımlılık kontrolü eksik.**
- **İyi | Yapısal loglama | ``Program.cs`` + `Serilog` — konsol + günlük dosya (14 gün saklama), EF Core/ASP.NET log seviyeleri kısılmış (disk dolumu sonrası düzeltilmiş). Correlation ID ise eksik.**
- **İyi | Drift detektörleri — Redis ve pazaryeri referans DB durumu açılışta tek satır log ile doğrulanıyor; konfigürasyon drift'i gizli yavaşlık olmaktan çıkarılmış.**

3.9 Canlı Ortam Ölçüm Bulguları (2026-08-16, düşük yük)

Prod **ecspros** servisi üzerinde sunucu-içi salt-okunur ölçüm + düşük RPS **curl**. Statik bulguları doğruluyor ve somut sayılar ekliyor:

Süreç / kaynak

- Prod API süreci: **active** (24 sa), **RSS 1.3 GB (zirve 1.7 GB)**, CPU ort. ~%7 (4 çekirdek, 9.7 GB RAM, yük ort. 0.23).
- Ayrıca **publish-staging** ikinci bir API süreci de çalışıyor (~356 MB) — aynı sunucuda iki instance toplam ~1.9 GB.

Redis (canlı)

- Açılışta **Redis cache: AKTİF ✓** (yaz-oku doğrulandı) — eski "Redis kullanılmıyor" bulgusu artık geçersiz.
- Aktif 5 anahtar, hepsi mağaza sıcak yolu: **channelcat:products:v11** (TTL ~90 sn), **channelcat:facets:v9** (~90 sn), **page:c5:homepage** (~280 sn), **page:c5:global-top** (2 segment).
- Bellek 1.5 MB, **evicted_keys=0**, reddedilen bağlantı 0 — cache sağlıklı; kısa TTL nedeniyle düşük yükte çoğu anahtar sürekli dolmuyor.

PostgreSQL

- 16.13, DB **2.2 GB**. **`pg\_stat\_statements` etkin değil** → yavaş sorgu kanıtı toplanamıyor (öneri: etkinleştir).
- En büyük tablolar: **product_variant_attributes 619 MB (~1.0 M satır)**, **erp_variant_data 469 MB**, **product_variants 249 MB**, **product_images 238 MB**, **product_attributes 133 MB**.

Uç gecikme (doğrudan API :5000, `curl`)

- Ana sayfa **/**: ~**11 ms**, 471 KB HTML (Redis hit — hızlı; HTML boyutu büyük).
- Ürün listesi **/urun-listesi**: **320 ms → 1.05 sn, 1.38 MB HTML** — **GetStoreProductsQuery** darboğazının canlı kanıtı.
- Ürün detayı **/urun/P-00023049**: ~**200 ms**, 519 KB HTML — 1 M satırlık **product_variant_attributes** üzerindeki kartezyen **Include** etkisi.
- Kurumsal **/hakkimizda**: ~7 ms; sepet **/sepet**: ~6-10 ms.
- Admin API (tokensiz): 401 (yetki doğru uygulanıyor). nginx :80 → 301 (HTTPS'e yönlendirme).

Log (24 sa): 2112 INF, 1 ERR, 0 WRN. Tek ERR: FluentValidation **ValidationException** "Unhandled exception" olarak loglanmış — **GlobalExceptionHandler** doğrulama hatasını 400 yerine **500'e** düşürüyor (canlıda doğrulandı; **ValidationException** switch'e eklenmeli).

4. Önceliklendirilmiş Öneriler

#	Öneri	Şiddet	Etki	Efor
---	-------	--------	------	------

1	Stok rezerve/tüket işlemlerini atomik yap (koşullu UPDATE ... WHERE quantity >= x veya pg_advisory_xact_lock + koşullu güncelleme; en azından IsConcurrencyToken)	Kritik	Fazla satış/kayıp güncelleme	Orta
2	Sırları izlenen dosyalardan çıkar, döndür, env/user-secrets'e taşı; commit geçmişinden temizle	Kritik	Kimlik bilgisi sızıntısı	Orta
3	Tüm DbContext'lere EnableRetryOnFailure ekle	Yüksek	Geçici DB hatası dayanıklılığı	Düşük
4	/health + health checks (DB/Redis) ekle	Yüksek	İzleme/LB/operasyon	Düşük
5	Domain event'lerini transaction/outbox'a al (en azından stok/ödeme akışları)	Yüksek	Kısmi durum/kayıp yan etki	Yüksek
6	Worker'lara distributed lock/leader election/SKIP LOCKED claim ekle	Yüksek	Çift yazma (para/e-posta/kargo)	Orta
7	Kartezyen Include'lere AsSplitQuery + read-only'a AsNoTracking	Yüksek	Detay sayfası yavaşlığı	Düşük-Orta
8	Kayıtsız named HttpClient'ları kaydet veya kaldır	Yüksek	Aktifleşince runtime crash	Düşük
9	Otomatik test altyapısı (en az stok/ödeme/sipariş akışları)	Yüksek	Regresyon güvencesi	Yüksek (sürekli)
10	Admin/supplier auth uçlarına app-layer rate limit	Orta	Brute-force	Düşük
11	SignalR Redis backplane + sticky session stratejisi	Yüksek (çoklu-instance için)	Realtime çoklu-instance	Orta
12	Data Protection anahtarlarını paylaşımlı depoya taşı + yedekle	Yüksek (çoklu-instance için)	Şifreli kimlik bilgisi	Orta
13	Cache geçersizleştirmeyi event-tabanlı yap (TTL-ağırlıklıdan çıkar) + stampede kilidi	Orta	Bayat veri / DB ani yük	Orta
14	Eski MD5/SHA256 üye hash'lerini zorla yeniden hash'le	Orta	Kırılabilir hash	Orta

15	Prod CORS'i daralt (localhost/HTTP origin'leri çıkar)	Orta	CORS yüzeyi	Düşük
----	---	------	-------------	-------

Kod optimizasyon raporunun P0 kazançları (Faz 2'ye işlendi): (1)

`GetStoreProductsQuery`'deki tüm-platform-fiyat yüklemesini sayfa-varyant batch'ine indirgemek; (2) bellekte sıralamayı PostgreSQL/read-model'e taşımak; (3) `PageComposer`'ı Redis hit yolunda DB'den bağımsızlaştırmak. Ayrıca: küçük kart DTO'su, blok batch/paralel çözme, stampede koruması, yayında pre-warm.

5. Fazlara Ayrılmış Düzeltme Yol Haritası

Faz 0 — Acil (günler): bugfix + sır hijyeni

Sıra: güvenlik ve veri doğruluğu en yüksek öncelik.

- Sırları taşı ve döndür** (`appsettings.json`, `docker-compose.yml`, 12x `*DbContextFactory.cs`):
- Ortak sırları environment variable / `dotnet user-secrets` / untracked `appsettings.Production.json`'a taşı.
- `docker-compose.yml`'de `${POSTGRES_PASSWORD}` / `${REDIS_PASSWORD}` kullan; değerleri `.env`'e (zaten gitignored) al.
- `*DbContextFactory.cs` design-time connection string'lerini env'den oku (veya secret-store).
- Değerleri **döndür** (eski değerler commit geçmişinde kaldığı için özellikle `Jwt:Secret` ve DB şifresi).
- `.gitignore`'ın `appsettings.json`'u (veya secret içeren bölümü) kapsadığından emin ol.
- Stok atomikliği** (`StockOps.cs`):
- Rezerve/tüket için `pg_advisory_xact_lock(variantId)` + koşullu `UPDATE ... WHERE Quantity >= x` deseni veya `Stock` entity'sine `RowVersion (IsConcurrencyToken)` ekle.
- Kabul testi: iki eşzamanlı checkout aynı son birimi satamaz.
- Kayıtsız named HttpClient'ları düzelt** — `visual-search`, `play-integrity`, `TrendyolSeller`, `TrendyolReference` için `AddHttpClient(name)` kayıtları ekle veya referansları kaldır. Timeout değerlerini de tanımla.
- ``Console.WriteLine`` ile **admin şifresi basmayı kaldır** (`DatabaseSeeder.cs`); varsayılan admin'i yalnızca Development'ta seed et.

Çıkış kriteri: Sırlar izlenen dosyalarda yok, stok güncellemesi atomik, gizli latent client kayıtları tamam.

Faz 1 — Dayanıklılık ve güvenlik temelleri (1-2 hafta)

- ``EnableRetryOnFailure()`` — tüm 15 modül `DependencyInjection.cs`'ine ekle (`MaxRetryCount`, `MaxRetryDelay`, hata kodu listesi).
- Health checks** — `AddHealthChecks()` + `MapHealthChecks("/health")`; DB ve Redis bağımlılık kontrolü; nginx `proxy_pass`'e `/health` probe ekle.
- Dış HTTP dayanıklılığı** — `AddResilientHttpClient`'ı `paytr`, SMS (GES Telekom), Integration adaptörleri ve SMTP için gerçekten kullan; her client'a açık timeout tanımla. (Polly helper'ı artık ölü kod olmaktan çıkar.)

4. **App-layer rate limit** — `AuthController` (login/token/refresh) ve `SupplierAuthController`'a `[EnableRateLimiting]` ekle.
5. **Eski üye hash'leri** — `MemberPasswordHasher`'da eski MD5/SHA256 doğrulamasını "süresi dolmuş, sıfırlama zorunlu" akışına bağla veya toplu yeniden hash'leme job'ı ekle.
6. **Prod CORS daraltma** — `localhost:3000/5173` ve düz HTTP origin'lerini prod'dan çıkar.
7. **Data Protection anahtar yedeği** — `~/ .ecsp/ dp-keys` yedekleme prosedürü doküman et ve otomatikleştir.
8. **Graceful shutdown** — `HostOptions.ShutdownTimeout` tanımla; worker'ların `stoppingToken` ile temiz durduğunu doğrula.

Çıkış kriteri: `/health` canlı; geçici DB/HTTP hataları otomatik toparlanıyor; admin/supplier girişi throttled; prod CORS dar.

Faz 2 — Performans (2-4 hafta)

Bu faz, [ECSPros-Kod-Optimizasyon-Raporu.pdf](#)'deki P0/P1/P2 önceliklerini ve 4 paketlik uygulama planını bu değerlendirmenin bulgularıyla birleştirir. Hedef bağlam: **4.000 eşzamanlı kullanıcı**, API/PostgreSQL/Redis'in **ayrı sunucularda** çalıştığı dağıtık yerleşim — bu durumda ana kaldıraç cache miktarı değil, **istek başına üretilen veri hacmi ve PostgreSQL ağ turu sayısıdır**.

Paket 1 — Ürün listeleme sıcak yolu (P0):

1. `GetStoreProductsQuery`'deki "tüm platform fiyatlarını yükleme" davranışını kaldır — `StorefrontChannelPricingService.GetActiveVariantPricesAsync`'i sayfadaki `variantIds` kümesiyle sınırla.
2. İki aşamalı listeleme: (a) filtre + sıralama + sayfalama → `ProductId` listesi + toplam; (b) yalnızca sayfadaki ID'ler için fiyat/stok/görsel/özellik/kampanya/yorum/sayaç toplu zenginleştirme.
3. Kart zenginleştirmeyi "ürün başına çağrı"dan "sayfa başına toplu çağrı"ya çevir (var olan `GetVariantAvailableStocksAsync/GetMinEffectivePricesAsync` deseni).

Paket 2 — Sıralama ve read-model (P0/P1):

1. Bellekte sıralamayı kaldır — sıralama/sayfalama `ORDER BY effective_price ... OFFSET/LIMIT` ile PostgreSQL'de yapılsın (tüm adayların `ToListAsync` ile belleğe alınmasını önler).
2. Efektif fiyat, puan, yorum, favori, sepet, görüntülenme, satış, stok durumu taşıyan **ürün arama read-model'ini** oluştur (uzun vadeli çözüm).
3. `GetProductDetail`, `GetOrderDetail`, `GetUserDetail`, `GetReturnDetail`, `GetFirmDetail`, `GetSessionSummary`, `ProductCampaignResolver`, `ApproveProductSubmission` için `AsSplitQuery()` + read-only'a `AsNoTracking()`.
4. Arama kelime döngüsü N+1'ini tek `IQueryable/predicate`'te birleştir; `Orders.MemberId`, `OrderItems.SupplierId`, `Members(Status, CreatedAt)` index'lerini ekle.

Paket 3 — Ana sayfa kompozisyonu ve cache (P0/P1):

1. `PageComposer` Redis hit yolundan DB'yi çıkar — aktif snapshot version'ını kısa ömürlü cache'te (`page-active:{platformId}`) tut; yayında güncelle.
2. Blokları batch / sınırlı paralellikte (2-3 eşzamanlı, ayrı DI scope + `DbContext`) çöz; `VitrinVmBuilder` koleksiyon-üye N+1'ini tek sorguya indir.
3. Cache stampede koruması: süreç-içi key-bazlı `SemaphoreSlim` + node'lar arası kısa Redis kilidi + stale-while-revalidate.

4. Cache invalidasyonu: ürün/fiyat/stok/kategori değişiminde `channelcat:*` anahtarlarını event ile temizle (TTL yedek güvence); `RemoveByPatternAsync`'i gerçek uygulamaya geçir veya deseni kaldır.

Paket 4 — Render ve payload (P1/P2):

1. Ana sayfa için küçük `StorefrontProductCardDto` (renk/beden/garanti/kampanya detayını detay sorgusuna bırak); kampanya (30-60 sn) ve sosyal sayaç (15-30 sn) TTL cache'i.
2. İlk HTML'i küçült (SSR ~422 KB ölçülmüş): ekran-altı blokları lazy-load, blok başına 8-12 ürün, inline script'leri ortak JS modülüne taşı, tekrarlanan JSON parse'ını azalt.
3. `InStockProductProvider`'i istenen `variantIds` ile sınırla (`GetInStockVariantIdsAsync`).

(Ölçüm + kabul): Her değişiklik öncesi/sonrası aynı veri setiyle P95/P99 ölçümü; soğuk/sıcak cache ana sayfa, kategori sayfaları, fiyat artan/azalan sıralama, çok kelimeli arama + renk eşleşmesi, kampanya + stoksuz ürün, Redis kısa süre kapalıyken doğru davranış, aynı cache anahtarına yüksek eşzamanlı istek senaryoları.

Çıkış kriteri: `GetStoreProductsQuery` artık tüm platform verisini yüklemiyor; sıralama DB/read-model'de; ana sayfa cache-hit'i DB'siz; p95/p99 ölçülmüş ve hedefe yakın; bayat cache penceresi bilinçli ve dokümente.

Faz 3 — Ölçeklenebilirlik / çoklu sunucu hazırlığı (4-8 hafta)

1. **Worker koordinasyonu** — tüm arka plan worker'larına ortak "claim/leader election" deseni (DB tabanlı `pg_try_advisory_lock` veya `SKIP LOCKED` claim; `MarketplaceReferenceSyncService` örnek alınır). Özellikle `SettlementEligibilityWorker`, `CargoNotifyWorker`, `LegacySyncWorker`, `SavedSearchNotifier`.
2. **SignalR backplane** — `AddStackExchangeRedis()` (SignalR Redis backplane) ekle; nginx'te WebSocket sticky (`ip_hash`) stratejisini belirle.
3. **Data Protection paylaşımı** — key ring'i Redis/key vault/sertifika tabanlı paylaşımli depoya taşı (`PersistKeysToRedis` veya `PersistKeysToFileSystem` paylaşımli dizin). Çoklu-makinede şifreli kimlik bilgilerinin ortak çözülmesi için zorunlu.
4. **Rate limiting dağıtık** — IP-bazlı limiter'ı Redis'e taşı (veya nginx'i tek otorite yapıp app-layer'ı kaldır).
5. **Açılış migration stratejisi** — migration/seed'i uygulamadan ayır (CI/deploy adımımda tek seferlik `dotnet ef database update`); açılışta `MigrateAsync` çağrılarını kaldır (özellikle CMS/CRM/Storefront).
6. **Medya depolama** — `LocalDiskImageUploadService`'i object storage/`IFormFile` → S3/NFS'ye taşı; statik medya için CDN.
7. **Kimlik güvenlik sayaçları** — üye giriş kilidi ve device-token secret'ını Redis'e taşı (dağıtık).
8. **Domain event/outbox** — stok/ödeme kritik akışlarını transactional outbox'a al (tablo + dispatcher); süreç-içi MediatR yayını yerine kalıcı kuyruk.
9. **(Ops)** İki instance'lı canary: ikinci instance'ı kısa süre aç, yukarıdakileri doğrula, tek instance'a geri dön.

Çıkış kriteri: 2+ instance arkasında LB ile çift yazma/çift bildirim yok; SignalR olayları tüm instance'lara ulaşıyor; şifreli kimlik bilgileri her instance'ta çözülüyor.

Faz 4 — Gözlemlenebilirlik ve test (sürekli)

1. **Correlation ID / trace** — `GlobalExceptionHandler` + request log'una trace-id ekle; Serilog enrichment.

2. **Yapılandırılmış hata modeli** — `ProblemDetails`, istisna filtreleme, hassas veri scrub.
3. **Otomatik test** — en kritik akışlardan başla: stok rezerve/tüket (fazla satış), sipariş numarası benzersizliği, hesap bakiyesi, kampanya indirim hesabı, kupon. `xUnit` + `Testcontainers` (PostgreSQL) ile entegrasyon testleri.
4. **Metrikler/APM** — istek süresi, DB sorgu süresi, cache hit/miss, kuyruk derinliği. (Health check endpoint'leri buraya beslenir.)
5. **Load testi CI'a** — kritik uçlarda regresyon eşiği.

6. Sonuç

ECSPros, **iş mantığı ve modülerlik açısından** tek instance'lık test/erken üretim için sağlam bir temel sunuyor. Ancak:

- **Veri doğruluğu** (stok atomikliği) ve **sır yönetimi** bugün yatırıma en çok değecek iki düzeltmedir; bunlar **Faz 0**'dadır.
- **Dayanıklılık** (DB retry, health check, HTTP dayanıklılığı, worker koordinasyonu) **Faz 1–3**'te sistematik olarak kapatılmalıdır.
- **Performans** iyileştirmeleri (`AsSplitQuery`, index, cache invalidasyonu) ölçülebilir kazanç sağlar ve **Faz 2**'dedir.
- **Çoklu sunucu** bugün bir seçenek değil; **Faz 3** tamamlanmadan 2+ instance'a çıkılmamalıdır.

Önerilen sıra: **Faz 0 → Faz 1 → (yük testi ile doğrula) → Faz 2 → Faz 3**, Faz 4'ü sürekli olarak paralel yürüt.

Bu belge statik analize ve **2026-08-16 canlı ortam ölçümüne** (§3.9) dayanır; tam yük testi (tepe yük, eşzamanlılık, p95/RPS) Faz 2 öncesi hâlâ tamamlayıcı olarak şarttır. Performans/ölçekleme açısından somut P0/P1/P2 düzeltme listesi ve dosya bazlı çalışma planı için [ECSPros-Kod-Optimizasyon-Raporu.pdf](#) ile birlikte okunmalıdır.